2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

Volume III: Visual Architecture Atlas

66 Original Technical Schematics, Packet Flows & Troubleshooting Decision Trees

Cloud Native Engineering Press

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

VOLUME III: THE KUBERNETES VISUAL ARCHITECTURE ATLAS

PART 1: CORE ARCHITECTURE & PACKET FLOW SCHEMATICS

(DIAGRAMS 01–32)

1. Complete Kubernetes Cluster Architecture

```
graph TB
    subgraph Control_Plane["CONTROL PLANE (MASTER)"]
        APIServer["kube-apiserver<br/>(Port 6443)"]
        Scheduler["kube-scheduler"]
        KCM["kube-controller-manager"]
        CCM["cloud-controller-manager"]
        ETCD[("etcd Cluster<br/>(Port 2379/2380)")]

        Scheduler -->|Watch / Bind| APIServer
        KCM -->|Reconcile State| APIServer
        CCM -->|Cloud Sync| APIServer
        APIServer <-->|Raft Consensus| ETCD
    end

    subgraph Worker_Node_1["WORKER NODE 1"]
        Kubelet1["kubelet (systemd)"]
        Proxy1["kube-proxy (iptables/IPVS)"]
        Runtime1["containerd (CRI Runtime)"]
        CNI1["CNI Plugin (veth / Bridge)"]
        Pod1["Pod A (10.244.1.10)"]
        Pod2["Pod B (10.244.1.11)"]

        Kubelet1 -->|gRPC / CRI| Runtime1
        Runtime1 --> Pod1
        Runtime1 --> Pod2
        CNI1 -.->|IP Allocation| Pod1
        CNI1 -.->|IP Allocation| Pod2
    end

    subgraph Worker_Node_2["WORKER NODE 2"]
        Kubelet2["kubelet (systemd)"]
        Proxy2["kube-proxy (iptables/IPVS)"]
        Runtime2["containerd (CRI Runtime)"]
        Pod3["Pod C (10.244.2.15)"]

        Kubelet2 -->|gRPC / CRI| Runtime2
        Runtime2 --> Pod3
    end

    Admin["Cluster Administrator / kubectl"] -->|HTTPS / mTLS| APIServer
    Kubelet1 -->|Heartbeat / Pod Status| APIServer
    Kubelet2 -->|Heartbeat / Pod Status| APIServer
    Proxy1 -->|Watch Endpoints| APIServer
    Proxy2 -->|Watch Endpoints| APIServer
```

Architectural Explanation:

The Kubernetes architecture divides cluster operations into the Control Plane and Worker Nodes. The `kube-apiserver` acts as the single central communications gateway; no component communicates directly with

`etcd` other than the API server. Controllers and schedulers run continuously, watching the desired state via API server stream watches and reconciling reality to match. Worker nodes run `kubelet` (managing container lifecycles via CRI) and `kube-proxy` (programming network filter rules via iptables or IPVS).

---

2. Control Plane Component Inter-Communication

```
sequenceDiagram
    autonumber
    actor Admin as Administrator (kubectl)
    participant API as kube-apiserver
    participant ETCD as etcd Store
    participant Sched as kube-scheduler
    participant KCM as kube-controller-mgr
    participant Kube as kubelet (Node)

    Admin->>API: POST /api/v1/namespaces/default/pods (Create Pod)
    API->>ETCD: Persist Pod Spec (nodeName: "")
    ETCD-->>API: Write Acknowledged
    API-->>Admin: 201 Created (Pod Pending)

    Note over API,Sched: Async Watch Stream Triggered
    API-->>Sched: Watch Event: Unassigned Pod Detected
    Sched->>Sched: Run Filtering (Predicates) & Scoring (Priorities)
    Sched->>API: POST /binding (Assign Pod to node-01)
    API->>ETCD: Update Pod Spec (nodeName: "node-01")
    ETCD-->>API: Write Acknowledged

    Note over API,Kube: Node Watch Stream Triggered
    API-->>Kube: Watch Event: Pod Scheduled to node-01
    Kube->>Kube: Request Runtime (CRI) & Network (CNI)
    Kube->>API: PATCH /status (Phase: Running, PodIP: 10.244.1.20)
    API->>ETCD: Persist Pod Status
```

Architectural Explanation:
All control-plane communication is asynchronous and mediated entirely by `kube-apiserver` watch streams. Neither the scheduler nor the kubelet poll; they maintain long-lived HTTP/2 streams watching specific API resource endpoints. When a scheduling decision is made, the scheduler writes a `/binding` subresource rather than mutating the Pod directly.

3. API Server Request Lifecycle

```
flowchart LR
    Request["Incoming HTTPS Request<br/>(Client / kubectl)"] --> AuthN{"1. Authentication<br/>(Cert, OIDC
    AuthN -- Rejected --> R401["401 Unauthorized"]
    AuthN -- Success --> AuthZ{"2. Authorization<br/>(RBAC, Node, Webhook)"}
    AuthZ -- Rejected --> R403["403 Forbidden"]
    AuthZ -- Success --> Mutating{"3. Mutating Webhooks<br/>(Modify Spec / Defaults)"}
    Mutating --> Schema{"4. Schema Validation<br/>(OpenAPI / Syntax)"}
    Schema -- Invalid --> R422["422 Unprocessable Entity"]
    Schema -- Valid --> Validating{"5. Validating Webhooks<br/>(Policy Enforcement)"}
    Validating -- Denied --> R400["400 Bad Request"]
    Validating -- Allowed --> ETCD[("6. etcd Storage<br/>(Write & Commit)")]
```

Architectural Explanation:
Every API request traverses a deterministic 6-stage pipeline:

1. **Authentication (AuthN):** Resolves client identity to Username, UID, and Groups using Client Certificates (mTLS), Bearer Tokens (ServiceAccounts), or OIDC.
2. **Authorization (AuthZ):** Evaluates RBAC `Roles` and `ClusterRoles` using Rule Evaluation (Node authorizer checks kubelet rights).
3. **Mutating Admission Webhooks:** Injects defaults or modifies the specification (e.g., Istio sidecar injection).
4. **Object Schema Validation:** Verifies schema definitions against OpenAPI v3 specifications.
5. **Validating Admission Webhooks:** Enforces cluster-wide constraints without mutating (e.g., Gatekeeper, Kyverno).
6. **etcd Persistence:** Object state is committed to disk via Raft consensus.

4. etcd Raft Distributed Consensus & Leader Election

```
stateDiagram-v2
    [*] --> Follower: Node Boot / Join
    Follower --> Candidate: Election Timeout Expires (150ms-300ms)
    Candidate --> Leader: Receives Majority Votes (Quorum: N/2 + 1)
    Candidate --> Follower: Discovers Current Leader or Newer Term
    Leader --> Follower: Steps Down if Higher Term Detected
```

Architectural Explanation:
etcd uses the Raft distributed consensus protocol. In an N-node cluster, a strict quorum of $\lfloor N/2 \rfloor + 1$ members is required to establish leadership and commit log entries:

- 3-node cluster: Quorum is 2 (tolerates 1 failure).

- 5-node cluster: Quorum is 3 (tolerates 2 failures). If quorum is lost, etcd refuses all write requests to prevent split-brain state corruption.

---

5. etcd Quorum Loss & Multi-Member Disaster Recovery

```
flowchart TD
    QuorumLoss["etcd Quorum Lost<br/>(2 of 3 Members Down)"] --> StopMembers["1. Stop kube-apiserver & et
    StopMembers --> InspectSnapshot["2. Identify Latest Valid Snapshot<br/>etcdctl snapshot status snapsh
    InspectSnapshot --> RestoreDataDir["3. Restore to New Data Directory<br/>etcdctl snapshot restore sna
    RestoreDataDir --> UpdateStaticPod["4. Update /etc/kubernetes/manifests/etcd.yaml<br/>Modify hostPath
    UpdateStaticPod --> KubeletRestart["5. Kubelet Automatically Recreates etcd Static Pod"]
    KubeletRestart --> VerifyHealth["6. Verify Health: etcdctl endpoint health<br/>Verify API Server: kub
```

Architectural Explanation:
When an etcd cluster experiences catastrophic failure, manual snapshot restoration is required. Restoring must always target a *new, non-existing directory* (`--data-dir=/var/lib/etcd-restored`) to prevent collision with corrupted wal files. The static pod manifest is then updated so kubelet restarts the container pointing to the clean database.

6. kube-scheduler Two-Phase Pipeline: Filtering & Scoring

```
flowchart TD
    UnscheduledPod["Unscheduled Pod Spec"] --> FilteringPhase["PHASE 1: FILTERING (Predicates)"]

    subgraph Filtering["Filtering Predicates (Drop Ineligible Nodes)"]
        F1["NodeResourcesFit (CPU / Memory Requests)"]
        F2["NodeName & NodeSelector Matching"]
        F3["PodFitsHostPorts (Port Collisions)"]
        F4["PodToleratesNodeTaints (Taint Check)"]
        F5["NodeAffinity & TopologySpread"]
    end

    FilteringPhase --> Filtering
    Filtering --> FilterEval{"Any Nodes Pass?"}
    FilterEval -- No --> PodPending["Pod Stuck in Pending<br/>(0/3 nodes available: Insufficient memory)"
    FilterEval -- Yes (1 Node) --> BindNode["Directly Bind to Only Passing Node"]
    FilterEval -- Yes (>1 Nodes) --> ScoringPhase["PHASE 2: SCORING (Priorities)"]

    subgraph Scoring["Scoring Priorities (Weight 0-100)"]
        S1["NodeResourcesBalancedAllocation"]
        S2["ImageLocalityPriority (Cached Image On Node)"]
        S3["NodeAffinityPriority (preferredDuringScheduling)"]
        S4["TaintTolerationPriority"]
    end

    ScoringPhase --> Scoring
    Scoring --> RankNodes["Calculate Weighted Sum Across All Plugins"]
    RankNodes --> SelectedNode["Node with Highest Cumulative Score Selected"]
    SelectedNode --> Bind["Write /binding Subresource to API Server"]
```

Architectural Explanation:

Scheduling operates in two distinct phases:

1. **Filtering (Predicates):** Evaluates hard requirements. If a node fails even one predicate (insufficient RAM, untolerated taint), it is removed from consideration.
2. **Scoring (Priorities):** Evaluates soft preferences on surviving nodes, assigning scores from 0 to 100. Weights are applied, and the node with the highest aggregate score is chosen.

7. kube-controller-manager Reconciliation Loops

```
flowchart LR
    DesiredState["Desired State<br/>(etcd Spec)"] --> Reconciler{"Reconciliation<br/>Control Loop"}
    ActualState["Actual State<br/>(Cluster Telemetry)"] --> Reconciler
    Reconciler -- Match --> Idle["Sleep & Watch"]
    Reconciler -- Deviation Detected --> Action["Execute Corrective Actions<br/>(Create / Terminate / Upd
    Action --> APICall["POST / DELETE to API Server"]
```

Architectural Explanation:
Every Kubernetes controller runs an infinite control loop (`syncHandler`). It computes the diff $\Delta = \text{Desired State} - \text{Actual State}$. If replicas desired is 5 and actual is 3, the Deployment controller instructs the ReplicaSet controller to issue API requests creating 2 new Pods.

8. kubelet Internal Architecture

```
graph TB
    subgraph Kubelet["kubelet Daemon (systemd)"]
        SyncLoop["syncLoop (Core Event Loop)"]
        PLEG["PLEG (Pod Lifecycle Event Generator)"]
        cAdvisor["cAdvisor (Metrics & Cgroup Telemetry)"]
        ProbeManager["Probe Manager (Liveness/Readiness)"]
        VolumeManager["Volume Manager (Attach & Mount)"]
    end

    APIServer["kube-apiserver"] <-->|Watch Pods / Report Status| SyncLoop
    PLEG -->|Runtime State Changes| SyncLoop
    cAdvisor -->|Node & Container Stats| SyncLoop
    ProbeManager -->|Health Telemetry| SyncLoop
    VolumeManager -->|Mount CSI Devices| SyncLoop

    SyncLoop -->|gRPC CRI| ContainerRuntime["containerd (/run/containerd/containerd.sock)"]
    SyncLoop -->|gRPC CNI| CNIPlugin["CNI Network Plugin (/opt/cni/bin)"]
    SyncLoop -->|gRPC CSI| CSIPlugin["CSI Node Plugin (/var/lib/kubelet/csi-plugins)"]
```

Architectural Explanation:
The kubelet is the primary node agent. The `syncLoop` orchestrates updates based on events from:

- Pod manifests (from API server watch, static pod directory `/etc/kubernetes/manifests/` , or HTTP endpoints).
- PLEG (Pod Lifecycle Event Generator), which queries the container runtime via CRI to detect stopped or crashed containers.
- Probe Manager, which executes HTTP, TCP, exec, or gRPC probes.

9. containerd Architecture, shim-v2, and runc

```
graph LR
    Kubelet["kubelet"] -->|CRI gRPC API| Containerd["containerd daemon<br/>(/run/containerd/containerd.so
    Containerd --> CtrShim["containerd-shim-v2<br/>(Per Pod Process)"]
    CtrShim --> Runc["runc (OCI Runtime)"]
    Runc --> Kernel["Linux Kernel<br/>(Namespaces, Cgroups, Chroot)"]
    Kernel --> Container["Running Container Process"]
    CtrShim -.->|Maintains STDIO & Exit Status| Container
```

Architectural Explanation:
`containerd` decouples pod execution from daemon restarts using `containerd-shim-v2`. When a container starts, `runc` initializes the Linux namespaces and cgroups, hands execution off to the containerized binary, and exits immediately. The lightweight shim stays alive as the container's parent process, holding open standard I/O pipes and capturing exit codes so `containerd` can restart without stopping running workloads.

10. Pod Lifecycle State Machine

```
stateDiagram-v2
    [*] --> Pending: Manifest Created
    Pending --> ContainerCreating: Scheduled to Node
    ContainerCreating --> Running: All InitContainers Completed & App Started
    Running --> CrashLoopBackOff: Container Exits with Non-Zero Code
    CrashLoopBackOff --> Running: Backoff Exponential Delay Elapsed (10s..300s)
    Running --> Succeeded: Run-To-Completion (Exit 0)
    Running --> Failed: Container Failed (restartPolicy: Never)
    Running --> Terminating: Deletion Request Received (SIGTERM -> Grace Period -> SIGKILL)
    Terminating --> [*]: Volume Unmounted & CNI Teardown
```

Architectural Explanation:
A Pod progresses through strict phases. `CrashLoopBackOff` is not an official phase but a condition indicating repeated container termination. Kubernetes applies an exponential backoff delay (10s, 20s, 40s... capped at 300s) before re-attempting initialization.

11. Multi-Container Pod Design Patterns

```
graph TD
    subgraph Pod["Pod Network & IPC Namespace (Shared localhost)"]
        Init["Init Container<br/>(Runs to completion first)"]
        App["Main Application Container<br/>(Port 8080)"]
        Sidecar["Sidecar Container<br/>(Log Shipper / Envoy Proxy)"]
        SharedVol["Shared Volume (emptyDir)<br/>/var/log/app"]
    end

    Init -->|1. Setup Configuration & DB Schema| SharedVol
    App -->|2. Writes Application Logs| SharedVol
    Sidecar -->|3. Tails Logs & Streams to Cloud| SharedVol
    Sidecar <-->|Localhost Intercept (127.0.0.1:8080)| App
```

Architectural Explanation:
Multi-container Pods share:

1. **Network Namespace:** Same IP address, port space, and `localhost` loopback interface.
2. **IPC Namespace:** Shared POSIX shared memory and semaphores.
3. **Mounted Volumes:** Containers share filesystem directories via `emptyDir` or shared PersistentVolumes. Init containers run sequentially to full exit code `0` before the main application and sidecars start in parallel.

12. Deployment, ReplicaSet, and Pod Cascading Hierarchy

```
graph TD
    Deploy["Deployment: web-app<br/>(spec.replicas: 3, strategy: RollingUpdate)"]
    RS1["ReplicaSet: web-app-v1 (hash: 6bf579)<br/>replicas: 0 (Historical)"]
    RS2["ReplicaSet: web-app-v2 (hash: 84d99c)<br/>replicas: 3 (Active)"]

    Pod1["Pod: web-app-v2-a"]
    Pod2["Pod: web-app-v2-b"]
    Pod3["Pod: web-app-v2-c"]

    Deploy -->|Owns & Manages| RS1
    Deploy -->|Owns & Manages| RS2
    RS2 -->|Owns| Pod1
    RS2 -->|Owns| Pod2
    RS2 -->|Owns| Pod3
```

Architectural Explanation:
Deployments do not create or manage Pods directly. A Deployment creates and manages `ReplicaSets` (identified by a 10-character pod-template-hash), and the ReplicaSets create and monitor the individual Pods. Rollbacks are executed simply by scaling up the old ReplicaSet while scaling down the current one.

13. Rolling Update Progression with maxSurge and maxUnavailable

```
sequenceDiagram
    autonumber
    participant D as Deployment (replicas: 4, maxSurge: 1, maxUnavailable: 1)
    participant V1 as RS v1 (Active: 4)
    participant V2 as RS v2 (New: 0)

    Note over D: Update Container Image Triggered
    D->>V2: Scale Up to 1 (Surge allowed: 4+1 = 5 total)
    V2-->>D: Pod 1 Running & Ready
    D->>V1: Scale Down to 3 (Unavailable allowed: 4-1 = 3 min ready)
    D->>V2: Scale Up to 2
    V2-->>D: Pod 2 Running & Ready
    D->>V1: Scale Down to 2
    D->>V2: Scale Up to 3
    V2-->>D: Pod 3 Running & Ready
    D->>V1: Scale Down to 1
    D->>V2: Scale Up to 4
    V2-->>D: Pod 4 Running & Ready
    D->>V1: Scale Down to 0
    Note over D: Rollout Fully Completed
```

Architectural Explanation:
Rolling updates preserve availability using two constraints:

- `maxSurge`: Maximum number of Pods that can be scheduled above the desired replica count.
- `maxUnavailable`: Maximum number of Pods that can be unavailable during the update. During an update, new Pods must pass readiness probes before older Pods are scheduled for termination.

14. StatefulSet Stable Network Identity & Ordered Scaling

```
graph TB
    subgraph StatefulSet["StatefulSet: db (replicas: 3)"]
        Pod0["db-0 (Ordinal 0)<br/>PV: data-db-0 (10Gi)"]
        Pod1["db-1 (Ordinal 1)<br/>PV: data-db-1 (10Gi)"]
        Pod2["db-2 (Ordinal 2)<br/>PV: data-db-2 (10Gi)"]
    end

    Headless["Headless Service: db-service (clusterIP: None)"]

    Headless -.->|DNS: db-0.db-service.default.svc.cluster.local| Pod0
    Headless -.->|DNS: db-1.db-service.default.svc.cluster.local| Pod1
    Headless -.->|DNS: db-2.db-service.default.svc.cluster.local| Pod2
```

Architectural Explanation:
StatefulSets provide:

1. **Deterministic Ordinals:** Pods are named `$(statefulset-name)-$(ordinal)` from 0 to $N-1$.
2. **Sequential Operations:** Pod i must be Running and Ready before Pod $i+1$ is initialized. Scale-down occurs in reverse order.
3. **Persistent Volume Attachment:** Each replica receives an independent PersistentVolume via `volumeClaimTemplates` that persists across pod deletion.
4. **Stable Network Identity:** Paired with a Headless Service (`clusterIP: None`), each Pod receives a predictable DNS A-record.

---

15. DaemonSet Node Scheduling & Taint Tolerations

```
graph TD
    subgraph Nodes["Cluster Nodes"]
        N1["Control Plane<br/>Taint: node-role.kubernetes.io/control-plane:NoSchedule"]
        N2["Worker Node 1<br/>No Taints"]
        N3["Worker Node 2 (Edge)<br/>Taint: dedicated=edge:NoSchedule"]
    end

    DS["DaemonSet: fluentd-logging"]

    DS -->|Tolerates control-plane taint| P1["fluentd Pod on Control Plane"]
    DS -->|Standard Scheduling| P2["fluentd Pod on Worker 1"]
    DS -.->|Blocked without edge toleration| N3
    P1 --> N1
    P2 --> N2
```

Architectural Explanation:
A DaemonSet ensures that a copy of a Pod runs on all (or selected) nodes. By default, DaemonSet pods automatically include tolerations for standard node taints (`node.kubernetes.io/not-ready`, `unreachable`, `disk-pressure`). To run on control-plane nodes or specialized nodes with custom taints, explicit tolerations must be defined in the DaemonSet pod template.

16. Job & CronJob Execution Pipeline

```
flowchart TD
    CronJob["CronJob: report-generator<br/>schedule: '0 2 * * *' (Daily 2 AM)"] --> TimeTrigger{"2:00 AM
    TimeTrigger --> CreateJob["Creates Job: report-generator-28491820"]
    CreateJob --> JobController["Job Controller Reconciler"]
    JobController --> LaunchPod["Launches Pod (restartPolicy: OnFailure)"]
    LaunchPod --> RunContainer["Executes Batch Workload"]
    RunContainer -- Exit 0 --> JobComplete["Job Status: Completed (Succeeded = 1)"]
    RunContainer -- Exit 1 --> CheckBackoff{"backoffLimit (e.g. 6) Exceeded?"}
    CheckBackoff -- No --> Reattempt["Exponential Backoff & Relaunch Pod"]
    CheckBackoff -- Yes --> JobFailed["Job Status: Failed"]
```

Architectural Explanation:
CronJobs manage time-based execution by instantiating `Jobs`. Jobs guarantee batch run-to-completion. Unlike Deployments, Pods managed by a Job must declare `restartPolicy: OnFailure` or `Never`. If a container fails, the Job controller retries until `backoffLimit` is reached.

17. ConfigMap and Secret Injection Mechanics

```
graph LR
    subgraph Storage["etcd Storage"]
        CM["ConfigMap: app-config<br/>(DATABASE_URL, LOG_LEVEL)"]
        SEC["Secret: app-secret<br/>(API_KEY, DB_PASS)"]
    end

    subgraph Pod["Target Pod"]
        Env["Environment Variables<br/>(env / envFrom)"]
        VolumeMount["Volume Mount<br/>/etc/config/app.properties<br/>/etc/secrets/token"]
    end

    CM -->|Injected at Pod Creation| Env
    SEC -->|Injected at Pod Creation| Env
    CM -->|Mounted as Files| VolumeMount
    SEC -->|Mounted as tmpfs Files| VolumeMount
```

Architectural Explanation:
ConfigMaps and Secrets can be consumed via:

1. **Environment Variables:** Values are evaluated at container creation. Modifying the ConfigMap *does not* update running environment variables without a pod restart.
2. **Volume Mounts:** Injected as files. Kubernetes atomic directory swapping ensures files are dynamically updated without restarting the pod (eventual consistency within kubelet sync periods). Secrets are stored in `tmpfs` (RAM) on the node to prevent disk persistence.

18. Resource Requests, Limits, and Kernel CFS Quotas

```
flowchart TD
    PodSpec["Pod Spec:<br/>resources.requests.cpu: 500m<br/>resources.limits.cpu: 1000m<br/>resources.req

    Translation --> CPUReq["cpu.shares = 512 (Relative Priority)"]
    Translation --> CPULim["cpu.cfs_quota_us = 100000<br/>cpu.cfs_period_us = 100000<br/>(Hard CFS Thrott
    Translation --> MemReq["memory.min / low (OOM Score Adjustment)"]
    Translation --> MemLim["memory.max = 536870912 Bytes (Hard Limit)"]

    CPULim --> CPUExceed{"App Exceeds 1000m CPU?"}
    CPUExceed -- Yes --> Throttled["Kernel Throttles CPU Execution Cycles<br/>(No Pod Kill)"]

    MemLim --> MemExceed{"App Exceeds 512Mi RAM?"}
    MemExceed -- Yes --> OOMKill["Linux Kernel OOM Killer Invoked<br/>Exit Code 137 (OOMKilled)"]
```

Architectural Explanation:

- **CPU Requests:** Used by `kube-scheduler` for placement and translated to kernel `cpu.shares`.
- **CPU Limits:** Enforced via Completely Fair Scheduler (CFS) bandwidth quotas. Exceeding limits results in CPU cycle throttling, not termination.
- **Memory Requests:** Determines QoS class (`Guaranteed`, `Burstable`, `BestEffort`) and OOM-score adjustment.
- **Memory Limits:** Enforced as a hard memory ceiling. Exceeding memory limits triggers an instantaneous kernel `OOMKilled` (Exit Code 137).

19. Liveness, Readiness, and Startup Probe Execution Lifecycle

```
flowchart TD
    Start["Container Started"] --> StartupCheck{"Startup Probe<br/>Defined?"}

    StartupCheck -- Yes --> RunStartup["Execute Startup Probe"]
    RunStartup -- Failing --> CheckStartupLimit{"failureThreshold<br/>Exceeded?"}
    CheckStartupLimit -- No --> RunStartup
    CheckStartupLimit -- Yes --> RestartStartup["Restart Container"]

    StartupCheck -- No / Succeeded --> ParallelChecks["Run Liveness & Readiness Concurrently"]

    subgraph ActiveChecks["Active Health Checking"]
        Readiness["Readiness Probe<br/>(Is app ready to receive traffic?)"]
        Liveness["Liveness Probe<br/>(Is app deadlocked / broken?)"]
    end

    ParallelChecks --> Readiness
    ParallelChecks --> Liveness

    Readiness -- Pass --> AddEndpoints["Add Pod IP to Service EndpointSlice<br/>(Traffic Flows)"]
    Readiness -- Fail --> RemoveEndpoints["Remove Pod IP from EndpointSlice<br/>(No Traffic Sent to Pod)"

    Liveness -- Pass --> MaintainRunning["Container Remains Running"]
    Liveness -- Fail --> KubeletRestart["Kubelet Kills Container & Triggers Restart"]
```

Architectural Explanation:

1. **Startup Probe:** Disables liveness and readiness checks during slow initialization. If it fails, the container is killed.
2. **Readiness Probe:** Controls service routing. A failed readiness probe removes the Pod from Service `EndpointSlices` without killing the container.
3. **Liveness Probe:** Detects deadlocks and internal application halts. A failed liveness probe triggers an immediate container restart by the kubelet.

20. Node Affinity, Anti-Affinity, and Topology Spread Constraints

```
graph TD
    Pod["Scheduled Pod Spec"]

    subgraph HardReq["Hard Requirements (RequiredDuringSchedulingIgnoredDuringExecution)"]
        NodeAffinity["Node Affinity:<br/>topology.kubernetes.io/zone In [us-east-1a, us-east-1b]"]
        PodAntiAffinity["Pod Anti-Affinity:<br/>MatchLabel: app=web (Do not co-locate with another web po
    end

    subgraph Topology["Topology Spread Constraints"]
        Spread["topologySpreadConstraints:<br/>maxSkew: 1<br/>topologyKey: topology.kubernetes.io/zone<br
    end

    Pod --> HardReq
    Pod --> Topology
    HardReq --> Filter["Filtering Phase: Drops non-matching nodes"]
    Topology --> Balance["Distributes replicas evenly across availability zones"]
```

Architectural Explanation:

- `requiredDuringSchedulingIgnoredDuringExecution`: Strict rule. If no node matches, the Pod remains `Pending`.
- `preferredDuringSchedulingIgnoredDuringExecution`: Soft rule used during the scoring phase.
- `topologySpreadConstraints`: Ensures high availability by preventing pods from concentrating in a single failure domain (zone, rack, or host), strictly bounding skew by `maxSkew`.

21. Taints, Tolerations, and Node Cordon/Drain Eviction

```
sequenceDiagram
    autonumber
    actor Admin as Administrator
    participant API as kube-apiserver
    participant Node as Worker Node (node-02)
    participant Pods as Workload Pods

    Admin->>API: kubectl cordon node-02
    API->>Node: Mark unschedulable (node.spec.unschedulable = true)
    Note over Node: No new pods will be scheduled on node-02

    Admin->>API: kubectl drain node-02 --ignore-daemonsets --delete-emptydir-data
    API->>Pods: Send Eviction API request to each Pod on node-02
    Note over API: Respects PodDisruptionBudgets (PDB)
    Pods->>Pods: Receive SIGTERM & Graceful Shutdown (30s)
    API->>Node: Delete Pods from node-02
    Note over API: Deployment Controller reschedules replacement Pods on other nodes

    Admin->>Node: Perform OS Patching / Kubelet Upgrade / Reboot
    Admin->>API: kubectl uncordon node-02
    API->>Node: Mark schedulable (node.spec.unschedulable = false)
```

Architectural Explanation:

- **Taints:** Applied to nodes (`key=value:Effect`). Effects include `NoSchedule` , `PreferNoSchedule` , and `NoExecute` .
- **Tolerations:** Declared on Pods to allow scheduling on tainted nodes.
- **Cordoning:** Marks a node unschedulable without disturbing running pods.
- **Draining:** Safely evicts running pods via the Eviction API (honoring PDBs) so the host can undergo maintenance.

22. Pod Priority, PriorityClasses, and Preemption

```
flowchart TD
    HighPod["Incoming High-Priority Pod<br/>(priorityClassName: production-critical, Value: 1000000)"] --
    Sched --> FitCheck{"Can Pod fit on any node<br/>without changes?"}
    FitCheck -- Yes --> ScheduleStandard["Schedule Normally"]
    FitCheck -- No --> Preemption["PREEMPTION ALGORITHM TRIGGERED"]

    Preemption --> ScanNodes["Scan nodes for lower-priority pods"]
    ScanNodes --> SelectVictims["Select lowest-priority victim pods<br/>(priorityClassName: batch-low, Va
    SelectVictims --> EvictVictim["Send Eviction / Delete to Victim Pods"]
    EvictVictim --> WaitGrace["Wait for termination grace period"]
    WaitGrace --> BindHigh["Bind High-Priority Pod to Freed Node Capacity"]
```

Architectural Explanation:
When high-priority workloads cannot be scheduled due to insufficient resources, the scheduler's preemption logic evicts lower-priority pods. The scheduler sets `nominatedNodeName` on the high-priority pod while waiting for the victims to terminate cleanly.

---

23. Horizontal Pod Autoscaler (HPA) Metrics Flow

```
sequenceDiagram
    autonumber
    participant App as Pods (App Replicas)
    participant cAd as cAdvisor (Node Kubelet)
    participant MS as metrics-server
    participant HPA as HPA Controller
    participant Dep as Deployment Controller

    App->>cAd: Execute Workload (CPU/Memory Consumption)
    cAd->>cAd: Aggregate Cgroup Telemetry
    MS->>cAd: Scrape Summary API (/stats/summary) every 15s
    HPA->>MS: Query Metrics API (metrics.k8s.io/v1beta1) every 15s
    Note over HPA: Compute Formula:<br/>desiredReplicas = ceil[currentReplicas * (currentMetric / targetM
    HPA->>Dep: Update Scale Subresource (e.g. Scale 2 -> 5 Replicas)
    Dep->>App: Spawn Additional Pods
```

Architectural Explanation:
The HPA controller queries the `metrics.k8s.io` API provided by `metrics-server`. It computes the target scale using the formula: $$\text{desiredReplicas} = \left\lceil \text{currentReplicas} \times \left(\frac{\text{currentMetricValue}}{\text{targetMetricValue}} \right) \right\rceil$$ Stabilization windows prevent rapid thrashing (flapping) during metric spikes.

24. ResourceQuota and LimitRange Admission Controller

```
flowchart LR
    CreateReq["kubectl create Pod Spec<br/>(cpu: 500m, memory: 1Gi)"] --> LR{"LimitRange Admission Check"

    subgraph LimitRangeRules["LimitRange (Per Pod/Container Defaults & Bounds)"]
        LR1["Inject default requests/limits if missing"]
        LR2["Enforce min/max allowed per container"]
    end

    LR --> LimitRangeRules
    LimitRangeRules -- Violates Bounds --> RejectLR["403 Forbidden: LimitRange Exceeded"]
    LimitRangeRules -- Passes --> RQ{"ResourceQuota Admission Check"}

    subgraph ResourceQuotaRules["ResourceQuota (Namespace Aggregate Capacity)"]
        RQ1["Sum current allocated requests + new request"]
        RQ2["Compare against hard quota ceiling"]
    end

    RQ --> ResourceQuotaRules
    ResourceQuotaRules -- Exceeds Aggregate --> RejectRQ["403 Forbidden: Exceeded Quota"]
    ResourceQuotaRules -- Under Ceiling --> Allow["Commit Pod to etcd"]
```

Architectural Explanation:

- **LimitRange:** Enforces constraints on individual Pods/containers within a namespace (min/max bounds and automatic default injection).
- **ResourceQuota:** Enforces aggregate limits on total resources (CPU, Memory, Pod count, PVC count) consumed across the entire namespace.

25. Kubernetes Networking: Intra-Node vs. Inter-Node Packet Flow

```
flowchart TD
    subgraph Node1["WORKER NODE 1 (192.168.1.10)"]
        PodA["Pod A<br/>10.244.1.5"] <-->|veth pair| Bridge1["cbr0 / CNI Bridge"]
        PodB["Pod B<br/>10.244.1.6"] <-->|veth pair| Bridge1
        Bridge1 <-->|Host Route| VXLAN1["VXLAN Tunnel Interface (flannel.1 / calico)"]
    end

    subgraph Node2["WORKER NODE 2 (192.168.1.20)"]
        PodC["Pod C<br/>10.244.2.8"] <-->|veth pair| Bridge2["cbr0 / CNI Bridge"]
        Bridge2 <-->|Host Route| VXLAN2["VXLAN Tunnel Interface (flannel.1 / calico)"]
    end

    PodA -.->|Intra-Node: Local Bridge Switching| PodB
    VXLAN1 == Encapsulated UDP Packet (Port 4789 / 8472) ==> VXLAN2
    VXLAN2 -->|Decapsulate| PodC
```

Architectural Explanation:

1. **Intra-Node:** Traffic between Pod A and Pod B stays entirely on the local Linux bridge (`cbr0` or veth pair). No encapsulation or external routing occurs.
2. **Inter-Node:** Traffic destined for Pod C (different subnet `10.244.2.0/24`) is routed to the overlay interface (e.g., `flannel.1`), encapsulated inside an outer UDP packet, transmitted across the physical network, and decapsulated by the destination node's CNI interface.

---

26. Kubernetes Service Routing: iptables vs. IPVS

```
graph TD
    Client["Client Pod (10.244.1.5)"] -->|Connects to Service ClusterIP 10.96.0.100:80| Kernel["Linux Ker

    subgraph Mode["kube-proxy Mode"]
        IPTables["iptables Mode (Sequential Chains)<br/>PREROUTING -> KUBE-SERVICES -> KUBE-SVC-XXX<br/>R
        IPVS["IPVS Mode (Kernel Hash Tables)<br/>Netlink IPVS Virtual Server (10.96.0.100:80)<br/>Algorit
    end

    Kernel --> Mode
    Mode --> DNAT["DNAT: Translate ClusterIP to Endpoint Pod IP (10.244.2.15:8080)"]
    DNAT --> TargetPod["Target Pod (10.244.2.15)"]
```

Architectural Explanation:
Services do not run as processes; they are virtual IPs programmed by `kube-proxy` .

- **iptables:** Uses sequentially evaluated packet filter chains. Rules grow $O(N)$ with service counts, introducing latency at scale.
- **IPVS:** Uses kernel hash tables offering $O(1)$ lookups, supporting large-scale enterprise deployments with granular load-balancing algorithms.

27. EndpointSlice Architecture

```
graph TD
    Svc["Service: api-service<br/>(ClusterIP: 10.96.10.50)"]

    subgraph Traditional["Legacy Endpoints (Single Monolithic Object)"]
        EP["Endpoints: api-service<br/>Contains 5,000 Pod IPs<br/>(1.5MB etcd object updated on ANY pod c
    end

    subgraph Modern["EndpointSlices (Scalable Partitioning)"]
        EPS1["EndpointSlice: api-service-1<br/>Endpoints 1 to 100"]
        EPS2["EndpointSlice: api-service-2<br/>Endpoints 101 to 200"]
        EPS3["EndpointSlice: api-service-N<br/>Endpoints N to N+100"]
    end

    Svc --> Traditional
    Svc --> Modern
```

Architectural Explanation:
Legacy `Endpoints` stored all backing pod IPs in a single API resource. For a service with 2,000 replicas, a single pod scaling event forced the API server and all kube-proxy instances to re-transmit the entire multi-megabyte object. `EndpointSlices` partition endpoints into chunks of 100, dramatically reducing control-plane bandwidth and CPU load.

28. Cluster DNS Resolution: CoreDNS

```
sequenceDiagram
    autonumber
    participant Pod as Application Pod
    participant CoreDNS as CoreDNS (10.96.0.10)
    participant Upstream as External Upstream DNS (8.8.8.8)

    Note over Pod: Read /etc/resolv.conf<br/>nameserver 10.96.0.10<br/>search default.svc.cluster.local s
    Pod->>CoreDNS: Query: backend-svc (Appends default.svc.cluster.local)
    CoreDNS->>CoreDNS: Match in-memory Service cache
    CoreDNS-->>Pod: Return Service ClusterIP (10.96.40.22)

    Pod->>CoreDNS: Query: external.example.com
    CoreDNS->>CoreDNS: Does not match cluster.local zone
    CoreDNS->>Upstream: Forward DNS Query to 8.8.8.8
    Upstream-->>CoreDNS: Return Public IP (93.184.216.34)
    CoreDNS-->>Pod: Return Public IP
```

Architectural Explanation:
Every Pod inherits `/etc/resolv.conf` configured with `nameserver 10.96.0.10` (CoreDNS Service IP) and

search paths. Queries for bare names (`backend-svc`) sequentially append search domains until a match is resolved by CoreDNS. External domains are forwarded upstream to host resolvers.

---

29. Ingress Controller Architecture

```
graph TD
    Client["External Client (HTTPS:443)"] --> LB["External Load Balancer"]
    LB --> NodePort["NodePort 30443 (Worker Nodes)"]
    NodePort --> IngressPod["Ingress Controller Pod (nginx / traefik)"]

    subgraph Rules["Ingress Resource (networking.k8s.io/v1)"]
        R1["Host: api.example.com /path: /v1 -> Service api-svc:8080"]
        R2["Host: web.example.com /path: /   -> Service web-svc:80"]
    end

    IngressPod --> Rules
    Rules --> PodA["api-svc Pod (10.244.1.15)"]
    Rules --> PodB["web-svc Pod (10.244.2.22)"]
```

Architectural Explanation:
An Ingress Controller is a specialized reverse proxy running inside the cluster. It bypasses `kube-proxy` ClusterIP routing for performance, querying `EndpointSlices` directly to stream traffic straight to container endpoints with TLS termination and path rewrites.

---

30. Gateway API Architecture & Role Separation

```
graph TD
    subgraph InfraProvider["Infrastructure Provider"]
        GC["GatewayClass: standard-gateway<br/>(controller: acme.io/gateway-controller)"]
    end

    subgraph ClusterAdmin["Cluster Administrator"]
        GW["Gateway: prod-gateway<br/>(Listeners: HTTPS:443, Host: *.example.com)"]
    end

    subgraph AppDeveloper["Application Developer (Namespaces)"]
        HR1["HTTPRoute: payments-route<br/>(ParentRef: prod-gateway, Path: /payments)"]
        HR2["HTTPRoute: orders-route<br/>(ParentRef: prod-gateway, Path: /orders)"]
        Svc1["Service: payments-svc"]
        Svc2["Service: orders-svc"]
    end

    GC -->|Defines Controller| GW
    GW -->|Attached By| HR1
    GW -->|Attached By| HR2
    HR1 --> Svc1
    HR2 --> Svc2
```

Architectural Explanation:
Gateway API succeeds Ingress by splitting routing into three decoupled personas:

1. **Infrastructure Provider (`GatewayClass`):** Defines the underlying load-balancing implementation.
2. **Cluster Administrator (`Gateway`):** Declares network ports, IP allocations, and TLS certificates.
3. **Application Developer (`HTTPRoute`):** Defines routing rules, header matches, and canary weights without needing cluster-admin privileges.

---

31. NetworkPolicy Traffic Filtering Matrix

```
graph LR
    subgraph External["Traffic Sources"]
        World["Internet / Outside Cluster"]
        OtherNS["Pods in Other Namespaces"]
        SameNS["Pods in Same Namespace"]
    end

    subgraph Target["Target Pod (app: secure-db, port: 5432)"]
        NetPol["NetworkPolicy Enforcement<br/>policyTypes: [Ingress, Egress]"]
        DBPod["Postgres Container (5432)"]
    end

    World -- Blocked by Default Deny --> NetPol
    OtherNS -- Allowed if namespaceSelector Matches --> NetPol
    SameNS -- Allowed if podSelector Matches app: backend --> NetPol
    NetPol --> DBPod
```

Architectural Explanation:

- By default, all Pod network traffic is **non-isolated** (open allow-all).
- The moment a `NetworkPolicy` selects a Pod, it transitions to **default-deny** for that policy type (Ingress or Egress).
- Traffic is whitelisted only when source criteria (podSelector, namespaceSelector, ipBlock) and destination port match explicitly.

32. CSI Architecture and Volume Mount Protocol

```
sequenceDiagram
    autonumber
    participant API as kube-apiserver
    participant Attacher as CSI External Attacher
    participant Provisioner as CSI External Provisioner
    participant Kube as Node Kubelet
    participant Driver as CSI Node Plugin

    API->>Provisioner: Create PVC with StorageClass
    Provisioner->>Provisioner: Provision Remote Storage Disk (e.g. AWS EBS / Ceph)
    Provisioner->>API: Create PersistentVolume (PV) bound to PVC

    API->>Attacher: Pod Scheduled to node-01
    Attacher->>Attacher: Call ControllerPublishVolume (Attach Disk to VM)
    Attacher->>API: Update VolumeAttachment Status (Attached)

    API->>Kube: VolumeAttachment Ready for Mount
    Kube->>Driver: NodeStageVolume (Format filesystem e.g. ext4)
    Kube->>Driver: NodePublishVolume (Bind mount to /var/lib/kubelet/pods/UID/volumes)
    Kube->>Kube: Launch Container with Mounted Filesystem
```

Architectural Explanation:
Container Storage Interface (CSI) separates storage plugins from core Kubernetes code:

1. **Controller Plugins:** Run on control plane (`csi-provisioner` , `csi-attacher`) to provision and attach block devices to nodes.
2. **Node Plugins:** Run as DaemonSets on worker nodes (`csi-node-driver-registrar`) to execute local format (`NodeStageVolume`) and mount (`NodePublishVolume`) calls.

VOLUME III: THE KUBERNETES VISUAL ARCHITECTURE ATLAS

PART 2: SYSTEMATIC TROUBLESHOOTING DECISION TREES (DIAGRAMS 33–64)

33. Universal 60-Second Node Triage Decision Tree

```
flowchart TD
    Start["Node Reports Issue / Alert"] --> Step1["1. Run: kubectl get nodes -o wide"]
    Step1 --> CheckStatus{"Is Node Status Ready?"}

    CheckStatus -- NotReady --> GoNotReady["Follow Diagram 34: Node NotReady Tree"]
    CheckStatus -- Ready --> Step2["2. Check Node Resource Pressure:<br/>kubectl describe node <name> | g

    Step2 --> PressureCheck{"Any Condition True?<br/>MemoryPressure, DiskPressure, PIDPressure"}
    PressureCheck -- Yes --> GoPressure["Follow Diagram 57-59: Resource Pressure Trees"]
    PressureCheck -- No --> Step3["3. Inspect System Logs on Node:<br/>journalctl -u kubelet -e --no-page

    Step3 --> KubeletLog{"Any Fatal Errors in Kubelet?"}
    KubeletLog -- Yes --> GoKubelet["Follow Diagram 35: Kubelet Systemd Failure"]
    KubeletLog -- No --> Step4["4. Check Container Runtime:<br/>crictl info && systemctl status container
    Step4 --> RuntimeCheck{"Runtime Active & Responsive?"}
    RuntimeCheck -- No --> GoRuntime["Follow Diagram 36: containerd Failure"]
    RuntimeCheck -- Yes --> NodeHealthy["Node Infrastructure Verified Healthy"]
```

Diagnostic Explanation:
When troubleshooting an affected cluster, never jump blindly into pod manifests. Follow this 60-second triage protocol to isolate node-level infrastructure failures from workload application faults in under one minute.

34. Node Status: NotReady Diagnostic and Remediation Algorithm

```
flowchart TD
    NotReady["Node Reports NotReady in kubectl get nodes"] --> SSH["SSH into Affected Node"]
    SSH --> SvcCheck["systemctl status kubelet"]

    SvcCheck --> IsRunning{"Is kubelet running?"}
    IsRunning -- Inactive / Failed --> StartKubelet["systemctl restart kubelet<br/>journalctl -u kubelet
    IsRunning -- Active (Running) --> CheckAPIReach{"Can Node reach kube-apiserver?<br/>curl -k https://<

    CheckAPIReach -- Connection Refused / Timeout --> NetIssue["Firewall / Network / Security Group Block
    CheckAPIReach -- 200 OK or 401 Unauthorized --> CertCheck{"Are Kubelet Certificates Valid?<br/>openss

    CertCheck -- Expired --> RenewCert["Regenerate Certs via kubeadm / kubeconfig bootstrap"]
    CertCheck -- Valid --> CgroupCheck{"Do Kubelet & containerd Cgroup Drivers Match?<br/>SystemdCgroup =

    CgroupCheck -- Mismatch --> FixCgroup["Align cgroup to systemd & restart daemons"]
    CgroupCheck -- Match --> DiskCheck{"Is Node Root Disk 100% Full?<br/>df -h /"}
    DiskCheck -- Full --> CleanDisk["Prune dead containers & logs: crictl rmi --prune"]
    DiskCheck -- Not Full --> NodeReady["Kubelet Posts NodeStatus -> Node Transitions to Ready"]
```

Diagnostic Explanation:
A `NotReady` condition occurs when the node lease fails to renew or heartbeat times out. Root causes fall into:

1. Dead `kubelet` systemd service.
2. Network partition to the API server on port 6443.
3. Expired TLS client certificates in `/var/lib/kubelet/pki/`.
4. Cgroup driver mismatch between kubelet and containerd.
5. Disk space exhaustion preventing container creation.

35. kubelet Systemd Service Failure & Cgroup Driver Mismatch

```
flowchart TD
    FailedKubelet["systemctl status kubelet reports failed"] --> Journal["Run: journalctl -u kubelet -n 1
    Journal --> Analyze{"Analyze Error Message"}

    Analyze -- "cgroup driver mismatch" --> CheckConfig["Inspect /etc/containerd/config.toml<br/>Look for
    CheckConfig --> EditConfig["Set SystemdCgroup = true<br/>systemctl restart containerd && systemctl re

    Analyze -- "failed to load kubelet config file" --> CheckFile["Inspect /var/lib/kubelet/config.yaml<b

    Analyze -- "failed to run Kubelet: misconfiguration: kubelet cgroup" --> SwapCheck{"Is Linux Swap Act
    SwapCheck -- Swap Active --> TurnOffSwap["Run: sudo swapoff -a<br/>Comment swap line in /etc/fstab"]

    Analyze -- "certificate signed by unknown authority" --> KubeconfigFix["Verify /etc/kubernetes/kubele
```

Diagnostic Explanation:
The kubelet will refuse to start if Linux swap memory is enabled (unless configured explicitly with `failSwapOn: false`) or if the cgroup driver does not match the container runtime (`systemd` is mandatory in modern Kubernetes).

---

36. Container Runtime (containerd) Socket and CRI Resolution

```
flowchart TD
    RuntimeDown["containerd service failing or crictl errors"] --> CheckStatus["systemctl status containe
    CheckStatus --> SockCheck{"Does socket exist?<br/>ls -la /run/containerd/containerd.sock"}

    SockCheck -- No --> CheckCrashLog["Inspect: journalctl -u containerd -n 50 --no-pager"]
    CheckCrashLog --> BrokenConfig{"Invalid syntax in /etc/containerd/config.toml?"}
    BrokenConfig -- Yes --> DefaultConfig["Generate clean config:<br/>containerd config default > /etc/co

    SockCheck -- Yes --> CrictlTest{"Does crictl work?<br/>crictl pods"}
    CrictlTest -- Socket error --> SetEndpoint["Configure /etc/crictl.yaml:<br/>runtime-endpoint: unix://
    CrictlTest -- Success --> Ready["Container runtime fully operational"]
```

Diagnostic Explanation:
`crictl` requires an explicit runtime endpoint configuration. If `/etc/crictl.yaml` is absent, run `crictl config --set runtime-endpoint=unix:///run/containerd/containerd.sock`. Corrupted `config.toml` files should be regenerated from `containerd config default`.

---

37. Control Plane Static Pod Outage & Log Recovery

```
flowchart TD
    ControlPlaneDown["Control Plane Not Responding<br/>kubectl fails: Connection Refused on 6443"] --> SS
    SSH --> InspectStatic["Inspect /etc/kubernetes/manifests/<br/>kube-apiserver.yaml, etcd.yaml, kube-sc

    InspectStatic --> CheckCRI["Inspect Static Containers with crictl:<br/>crictl ps -a --name kube-apise
    CheckCRI --> ContainerFound{"Is API Server container listed?"}

    ContainerFound -- Exited / Crashing --> FetchLogs["Fetch logs directly from container runtime:<br/>cr
    ContainerFound -- Missing Entirely --> ManifestCheck["Verify manifest syntax & permissions in /etc/ku

    FetchLogs --> ErrorType{"Identify Root Error"}
    ErrorType -- "etcdserver: request timed out / connection refused" --> CheckEtcd["Verify etcd static c
    ErrorType -- "address already in use" --> PortConflict["Kill orphaned process holding port 6443: fuse
    ErrorType -- "certificate has expired" --> CertRenew["Renew certificates: kubeadm certs renew all"]
```

Diagnostic Explanation:
When the API server is down, `kubectl` cannot be used. Troubleshooting must be conducted on the control-

plane host using `crictl ps -a` and inspecting raw static pod container logs directly under `/var/log/pods/` or via `crictl logs`.

---

38. kube-apiserver Port 6443 Connection Refused Resolution

```
flowchart TD
    ConnRefused["kubectl: The connection to server 192.168.1.10:6443 was refused"] --> Step1["Check kubel
    Step1 -- Inactive --> StartK["systemctl start kubelet"]
    Step1 -- Active --> Step2["Check if apiserver container is running:<br/>crictl ps | grep kube-apiserv

    Step2 -- Running --> FWCheck["Check Linux Firewall / iptables:<br/>ufw status or iptables -L -n | gre
    FWCheck -- Blocked --> AllowPort["Allow TCP Port 6443"]

    Step2 -- Not Running --> Step3["Inspect latest apiserver container log:<br/>crictl logs $(crictl ps -
    Step3 --> AnalyzeErr{"Error Cause?"}
    AnalyzeErr -- ETCD unreachable --> FixETCD["Recover etcd container on port 2379"]
    AnalyzeErr -- Bad flag in manifest --> FixYAML["Fix invalid flag in /etc/kubernetes/manifests/kube-ap
    AnalyzeErr -- Expired PKI --> FixPKI["kubeadm certs renew all && systemctl restart kubelet"]
```

Diagnostic Explanation:
"Connection refused" on port 6443 means no process is listening on the port. It is almost always caused by the `kube-apiserver` static pod crashing on startup due to etcd connection failure or bad arguments in `/etc/kubernetes/manifests/kube-apiserver.yaml`.

---

39. etcd Cluster Health, Cert Expiration & Quorum Restoration

```
flowchart TD
    ETCDAlert["etcd Health Alert / Timeout"] --> EnvSetup["Export Auth Environment Variables:<br/>ETCDCTL_

    EnvSetup --> HealthCheck["Run: etcdctl --cacert=$CACERT --cert=$CERT --key=$KEY<br/>--endpoints=https

    HealthCheck --> IsHealthy{"Is endpoint healthy?"}
    IsHealthy -- Yes --> CheckQuorum["Run: etcdctl member list -w table<br/>Verify all members have activ

    IsHealthy -- No (tls: bad certificate) --> CheckCerts["Check cert dates:<br/>kubeadm certs check-expi
    CheckCerts --> RenewAll["kubeadm certs renew etcd-server etcd-peer"]

    IsHealthy -- No (connection refused) --> CheckDisk["Check if etcd disk is full (df -h /var/lib/etcd)"
    CheckDisk -- Out of space --> ExpandDisk["Free disk space or restore from uncorrupted snapshot"]
```

Diagnostic Explanation:
All `etcdctl` commands in Kubernetes require mTLS authentication using `/etc/kubernetes/pki/etcd/server.crt` and `server.key`. Always verify cluster health before attempting snapshot restores.

40. CoreDNS Resolution Failure & Loop Detection Diagnostic Tree

```
flowchart TD
    DNSFail["Pod cannot resolve service names (e.g. nslookup kubernetes.default)"] --> Step1["Check CoreD

    Step1 --> PodState{"CoreDNS Pods Running?"}
    PodState -- CrashLoopBackOff --> CheckLoop{"Are logs reporting 'Loop ... detected'?"}
    CheckLoop -- Yes --> FixLoop["Loop occurs when host /etc/resolv.conf points to 127.0.0.53 loopback.<b

    PodState -- Pending --> CheckSched["Check node taints or missing CNI network plugin"]

    PodState -- Running --> Step2["Test DNS Service IP & Endpoints:<br/>kubectl get svc kube-dns -n kube-

    Step2 --> EndpointsPresent{"Are endpoints populated?"}
    EndpointsPresent -- No --> CheckKubeProxy["kube-proxy issue or pod selector mismatch"]
    EndpointsPresent -- Yes --> Step3["Test Direct Query:<br/>kubectl run test-dns --rm -it --image=busyb
    Step3 -- Success --> NetPolCheck["Inspect NetworkPolicies in application namespace blocking Port 53 U
```

Diagnostic Explanation:
CoreDNS CrashLoopBackOff with `Loop detected` is a classic Linux systemd-resolved bug where CoreDNS forwards to local loopback and detects infinite recursion. The fix is updating host nameservers and editing the `Corefile` in `ConfigMap/coredns`.

41. Pod Stuck in Pending: Scheduling Failure Flowchart

```
flowchart TD
    PendingPod["Pod Stuck in Pending"] --> Describe["Run: kubectl describe pod <pod-name>"]
    Describe --> ReadEvents["Examine 'Events' Section at Bottom"]

    ReadEvents --> Reason{"Evaluate Scheduling Failure Reason"}

    Reason -- "0/N nodes available: Insufficient cpu / memory" --> Action1["Lower pod resource requests o
    Reason -- "node(s) had untolerated taint" --> Action2["Add matching toleration to pod spec or remove
    Reason -- "node(s) didn't match Pod's node affinity/selector" --> Action3["Correct labels on node (ku
    Reason -- "0/N nodes available: node(s) had volume node-affinity conflict" --> Action4["PVC is bound
    Reason -- "No events recorded" --> Action5["kube-scheduler is dead or control plane disconnected"]
```

Diagnostic Explanation:
If a Pod is `Pending`, the `kube-scheduler` has failed to bind it to a node. The `Events` table of `kubectl describe pod` lists the exact predicate that failed on every node.

42. Pod CrashLoopBackOff: Exit Code Analysis

```
flowchart TD
    Crash["Pod in CrashLoopBackOff"] --> GetCode["Run: kubectl describe pod <pod-name><br/>Inspect Last S

    GetCode --> ExitCode{"Analyze Exit Code"}

    ExitCode -- "Exit Code 0" --> Reason0["Container completed work and exited normally.<br/>Fix: Add lon
    ExitCode -- "Exit Code 1" --> Reason1["Application general error / syntax error.<br/>Check: kubectl l
    ExitCode -- "Exit Code 137" --> Reason137["OOMKilled (Out of Memory - 128 + 9 SIGKILL).<br/>Fix: Incr
    ExitCode -- "Exit Code 139" --> Reason139["Segmentation Fault (SIGSEGV). Corrupted binary or library.
    ExitCode -- "Exit Code 143" --> Reason143["Graceful Termination (128 + 15 SIGTERM). Stopped by orches
    ExitCode -- "Exit Code 255" --> Reason255["Entrypoint or CMD command does not exist in container imag
```

Diagnostic Explanation:
Exit codes convey the precise termination cause:

- Exit Code 0: Task finished (Jobs use this; long-running services crash without a persistent foreground daemon).
- Exit Code 137: Memory limit exceeded (OOMKilled) or external `kill -9`.
- Always run `kubectl logs <pod-name> --previous` to see what printed immediately before the crash.

43. ImagePullBackOff & Private Registry Credential Failure

```
flowchart TD
    ImageErr["Pod in ImagePullBackOff / ErrImagePull"] --> Describe["Run: kubectl describe pod <pod-name>
    Describe --> CheckMsg{"Analyze Failed Event Message"}

    CheckMsg -- "manifest for <image>:<tag> not found" --> FixTag["Image tag does not exist. Correct tag
    CheckMsg -- "repository does not exist or may require 'docker login'" --> FixAuth["Private Registry A
    CheckMsg -- "dial tcp: lookup registry... i/o timeout" --> FixDNS["Worker node cannot resolve DNS or

    FixAuth --> CreateSecret["Create Docker Registry Secret:<br/>kubectl create secret docker-registry re
    CreateSecret --> AttachSecret["Add imagePullSecrets to Pod Spec:<br/>spec.imagePullSecrets:<br/>  - n
    AttachSecret --> PodPulls["Kubelet Authenticates & Pulls Image"]
```

Diagnostic Explanation:
`ImagePullBackOff` indicates that the container image cannot be downloaded. Ensure the image tag is exact and that `imagePullSecrets` are defined in the pod manifest or bound to the ServiceAccount.

44. CreateContainerConfigError & Missing ConfigMap/Secret

```
flowchart TD
    ConfigErr["Pod in CreateContainerConfigError"] --> Describe["Run: kubectl describe pod <pod-name>"]
    Describe --> InspectMsg{"Analyze Event Message"}

    InspectMsg -- "configmap '<name>' not found" --> CreateCM["Create missing ConfigMap in the SAME names
    InspectMsg -- "secret '<name>' not found" --> CreateSec["Create missing Secret in the SAME namespace
    InspectMsg -- "key '<key>' not found" --> AddKey["Add missing key-value pair to ConfigMap or Secret"]

    CreateCM --> KubeletAutoStarts["Kubelet automatically detects created resource and starts Pod"]
    CreateSec --> KubeletAutoStarts
    AddKey --> KubeletAutoStarts
```

Diagnostic Explanation:
`CreateContainerConfigError` occurs before the container process is launched because a referenced `configMapKeyRef` or `secretKeyRef` does not exist in the pod's namespace. The pod transitions to `Running` automatically once the missing resource is created.

45. Service Connectivity Failure: ClusterIP Unreachable

```
flowchart TD
    SvcFail["Cannot reach Service ClusterIP (curl http://<cluster-ip>:<port>)"] --> Step1["1. Verify Endp

    Step1 --> HasEndpoints{"Are Pod IPs listed under ENDPOINTS?"}
    HasEndpoints -- None / <none> --> CheckSelector["Compare Service spec.selector with Pod metadata.labe
    CheckSelector --> CheckProbes["Check if backing Pods are failing Readiness probes.<br/>Unready pods a

    HasEndpoints -- Valid Pod IPs --> Step2["2. Test direct Pod IP connectivity:<br/>curl http://<pod-ip>
    Step2 -- Direct Pod Fails --> CheckApp["App inside container not listening on expected port or bound

    Step2 -- Direct Pod Succeeds --> Step3["3. Inspect kube-proxy & iptables:<br/>kubectl get pods -n kub
    Step3 --> ProxyHealthy{"kube-proxy running on all nodes?"}
    ProxyHealthy -- No --> RestartProxy["Restart kube-proxy daemonset"]
    ProxyHealthy -- Yes --> NetPolCheck["Check for NetworkPolicies blocking ingress on target pod"]
```

Diagnostic Explanation:
A healthy Service requires:

1. Exact label selector matching pod labels.
2. Backing pods passing readiness probes.
3. Target container listening on `0.0.0.0` (not `127.0.0.1`).
4. Active `kube-proxy` iptables/IPVS rules on every node.

46. NodePort Service Port Refused & Firewall Mismatch

```
flowchart TD
    NPFail["Cannot reach NodePort (curl http://<node-ip>:<nodePort>)"] --> Step1["Check Service Definitio
    Step1 --> VerifyType{"Is service type NodePort?"}
    VerifyType -- No --> FixType["Set spec.type: NodePort"]

    VerifyType -- Yes --> Step2["Identify allocated NodePort (Range 30000-32767)"]
    Step2 --> LocalTest["SSH into Node & test locally:<br/>curl http://127.0.0.1:<nodePort>"]

    LocalTest -- Works on Node --> ExternalFW["Local node port works -> External firewall / Cloud Securit
    LocalTest -- Connection Refused --> ProxyCheck["Check kube-proxy logs for iptables programming errors
```

Diagnostic Explanation:
If a NodePort works via `curl http://127.0.0.1:<port>` on the node itself but fails from outside, the host firewall (`ufw` , `firewalld`) or cloud security group is blocking the port range 30000–32767.

47. Ingress 404 Not Found & 502/503 Bad Gateway

```
flowchart TD
    IngressErr["HTTP Error via Ingress"] --> Code{"HTTP Status Code?"}

    Code -- 404 Not Found --> CheckHostPath["Verify Ingress Host & Path:<br/>1. Does request Host header

    Code -- 502 Bad Gateway --> CheckTarget["Target backend Service / Pod is refusing connection.<br/>Ver

    Code -- 503 Service Unavailable --> CheckEndP["Service has 0 healthy endpoints.<br/>Check pod readine
```

Diagnostic Explanation:

- **404 Not Found:** Ingress controller received the request, but no rule matched the `Host` or `Path` .
- **502 Bad Gateway:** Ingress routed traffic to the upstream pod, but the pod refused the TCP connection (port mismatch).
- **503 Service Unavailable:** No backing pods are in `Ready` state.

48. Gateway API HTTPRoute Detached & Unresolved ParentRef

```
flowchart TD
    RouteFail["HTTPRoute Not Serving Traffic"] --> InspectRoute["Run: kubectl describe httproute <name>"]
    InspectRoute --> CheckConditions["Check Status -> Conditions"]

    CheckConditions --> ParentRefCond{"Accepted: True & Programmed: True?"}

    ParentRefCond -- "Accepted: False / ResolvedRefs: False" --> RefIssue{"Why is ParentRef Rejected?"}
    RefIssue -- "Gateway not found" --> CheckGWName["Verify spec.parentRefs[].name matches an existing Ga
    RefIssue -- "Cross-namespace reference not allowed" --> ReferenceGrant["Create ReferenceGrant in Gate
    RefIssue -- "SectionName / Port mismatch" --> FixPort["Ensure HTTPRoute parentRef sectionName matches

    ParentRefCond -- "Accepted: True" --> BackendCheck["Verify backendRefs point to valid Services with e
```

Diagnostic Explanation:
Gateway API routes decouple routing from gateways. An `HTTPRoute` must be accepted by the parent `Gateway`. If they reside in different namespaces, a `ReferenceGrant` resource is mandatory for security authorization.

49. NetworkPolicy Dropped Packet Triage

```
flowchart TD
    NetPolIssue["Pods Cannot Communicate / Timeouts Occur"] --> CheckCNI["1. Verify CNI Supports NetworkP

    CheckCNI -- Flannel Only --> UpgradeCNI["Install Calico or Canal to enforce NetworkPolicies"]
    CheckCNI -- Supported --> Step2["2. Inspect Policies in Namespace:<br/>kubectl get netpol -n <namespa

    Step2 --> EvaluateRules["Evaluate podSelector and policyTypes:<br/>Does any NetPol select the target
    EvaluateRules -- Selected --> CheckDefaultDeny["Selected pod is now in Default-Deny mode for that tra

    CheckDefaultDeny --> VerifyWhitelist["Verify Ingress/Egress Whitelist Rules:<br/>1. podSelector match
    VerifyWhitelist -- Mismatch --> FixNetPol["Update NetworkPolicy YAML to permit required traffic"]
```

Diagnostic Explanation:
NetworkPolicies are silent drop filters. If `flannel` is used alone, policies are silently ignored without error. When debugging, test reachability with `kubectl exec -it <pod> -- nc -zvw3 <ip> <port>`.

50. PersistentVolume & Claim Stuck in Pending / Binding Failure

```
flowchart TD
    PVCFail["PVC Stuck in Pending"] --> DescribePVC["Run: kubectl describe pvc <pvc-name>"]
    DescribePVC --> CheckEvents["Read Events Section"]

    CheckEvents --> EventMsg{"Analyze Event Message"}

    EventMsg -- "waiting for first consumer to be created" --> NormalBehavior["StorageClass uses volumeBi

    EventMsg -- "no persistent volumes available" --> StaticIssue["Static Provisioning: No existing PV ma
    StaticIssue --> FixPV["Create PV with capacity >= PVC request and identical accessModes"]

    EventMsg -- "provisioning failed" --> DynamicIssue["Dynamic Provisioning: CSI driver error or invalid
```

Diagnostic Explanation:
`volumeBindingMode: WaitForFirstConsumer` is standard practice. It delays volume creation until the Pod is placed on a node, ensuring the storage volume is provisioned in the same availability zone as the node.

51. Volume Mount Failure: Multi-Attach Error & Node Lock

```
flowchart TD
    AttachFail["Pod in ContainerCreating: Volume Attach Failed"] --> Describe["Run: kubectl describe pod
    Describe --> ReadErr{"Multi-Attach error for volume ... Volume is already exclusively attached to one

    ReadErr --> Cause["Previous node did not release disk lock during sudden crash / cordon"]
    Cause --> Step1["1. Identify old node holding attachment:<br/>kubectl get volumeattachments"]
    Step1 --> Step2["2. Delete orphaned VolumeAttachment object:<br/>kubectl delete volumeattachment <att
    Step2 --> Step3["3. Alternatively, force delete terminated pod on old node:<br/>kubectl delete pod <o
    Step3 --> VolumeMounts["Kubelet attaches volume to new node & starts Pod"]
```

Diagnostic Explanation:
Cloud block storage (EBS, GPD, Cinder) can only attach to one VM at a time (`ReadWriteOnce`). When a node crashes, the `csi-attacher` waits for safe detachment before allowing a new host to mount the volume.

52. Dynamic Storage Provisioner Failure & StorageClass Tree

```
flowchart TD
    DynFail["Dynamic PVC Provisioning Fails"] --> SC["Inspect StorageClass: kubectl get sc <name> -o yaml
    SC --> CheckDriver["Check provisioner field (e.g. ebs.csi.aws.com, standard)"]

    CheckDriver --> CheckDriverPods["Verify CSI Controller Pods are Running:<br/>kubectl get pods -A | gr
    CheckDriverPods -- CSI Pods Down --> FixCSI["Restart CSI controller deployment"]
    CheckDriverPods -- Running --> CheckLogs["Inspect CSI provisioner logs:<br/>kubectl logs -n kube-syst
    CheckLogs --> IAMAuth["Cloud IAM permission denied / invalid disk type parameter in StorageClass"]
```

Diagnostic Explanation:
Dynamic volume provisioning depends on the in-cluster CSI storage controller. If PVC provisioning fails without events, inspect the `csi-provisioner` container logs in `kube-system`.

53. RBAC Unauthorized (403 Forbidden) Audit

```
flowchart TD
    RBACFail["API Error: 403 Forbidden: User 'X' cannot 'verb' 'resource' in namespace 'Y'"] --> Step1["T

    Step1 --> CheckRole["Inspect Role / ClusterRole:<br/>Does any rule grant verb on resource?"]
    CheckRole -- Missing Verb/Resource --> UpdateRole["Add required apiGroups, resources, and verbs to Ro

    CheckRole -- Rule Exists --> CheckBinding["Inspect RoleBinding / ClusterRoleBinding"]
    CheckBinding --> CheckSubject{"Does subjects[] match User/Group/ServiceAccount exactly?"}
    CheckBinding -- Namespace Mismatch --> FixBinding["Ensure ServiceAccount namespace in subjects matche
    CheckBinding -- Typo in Name --> FixSubject["Correct user/group/serviceaccount name"]
```

Diagnostic Explanation:
Always use `kubectl auth can-i <verb> <resource> --as=<user> -n <ns>` to verify permissions without switching contexts. Common errors include forgetting `apiGroups` (e.g., `apps` for deployments) or referencing a ServiceAccount in the wrong namespace in `subjects`.

54. Certificate Expiration & kubeadm certs Renewal

```
flowchart TD
    CertAlert["Cluster Down / Cert Expiration Warning"] --> CheckDates["Run: kubeadm certs check-expirati
    CheckDates --> ScanTable["Inspect Expiration Date for all Control Plane Certificates"]

    ScanTable --> Expired{"Any Certificate Expired?"}
    Expired -- Yes --> RenewCommand["Run: kubeadm certs renew all"]
    RenewCommand --> RestartPods["Restart Control Plane Static Pods:<br/>Move manifests temporarily out o
    RestartPods --> UpdateAdminConf["Update local admin.conf:<br/>cp /etc/kubernetes/admin.conf ~/.kube/c
    UpdateAdminConf --> Verify["Verify kubectl get nodes succeeds"]
```

Diagnostic Explanation:
Kubernetes control plane certificates created by `kubeadm` expire after 1 year. The renewal process takes less than 2 minutes using `kubeadm certs renew all`, followed by reloading the static pods.

55. Cluster Upgrade Failure & Node Version Skew Recovery

```
flowchart TD
    UpgradeFail["kubeadm upgrade plan or apply fails"] --> CheckError{"Analyze Upgrade Error"}

    CheckError -- "version skew violation" --> SkewRule["Kubernetes requires upgrading minor versions seq

    CheckError -- "control plane not ready" --> HealthCheck["Ensure all nodes are Ready and etcd is healt

    CheckError -- "apt-get / package hold error" --> Unhold["Unhold packages: sudo apt-mark unhold kubead

    Unhold --> Reapply["Run: sudo kubeadm upgrade apply v1.31.2"]
```

Diagnostic Explanation:
Upgrades must follow strict sequence: Control Plane first (`kubeadm upgrade apply` , then upgrade `kubelet`), followed by worker nodes one-by-one (`kubeadm upgrade node` , drain, upgrade `kubelet` , uncordon).

56. Worker Node Join Failure: Token & Discovery Timeout

```
flowchart TD
    JoinFail["kubeadm join times out or throws token error"] --> CheckMsg{"Analyze Join Error"}

    CheckMsg -- "token has expired" --> CreateToken["Generate new join token on Control Plane:<br/>kubead

    CheckMsg -- "discovery token CA hash mismatch" --> GetHash["Compute correct CA hash on Control Plane:

    CheckMsg -- "connection refused / timeout to 6443" --> TestNet["Test network connectivity from worker

    CreateToken --> RunJoin["Execute corrected kubeadm join command on worker"]
```

Diagnostic Explanation:
Bootstrap tokens expire after 24 hours. If joining a worker node after token expiration, generate a fresh join command using `kubeadm token create --print-join-command`.

57. High CPU & Memory Throttling: CFS Quota Investigation

```
flowchart TD
    Throttle["Application Latency Spikes / High CPU Alarm"] --> Step1["Check Pod Resource Metrics:<br/>ku
    Step1 --> Step2["Check Node Resource Metrics:<br/>kubectl top nodes"]

    Step2 --> CheckCgroup["SSH to Worker Node & Inspect CFS Throttling:<br/>cat /sys/fs/cgroup/cpu/kubepo
    CheckCgroup --> MetricEval{"nr_throttled > 0 and throttled_time is high?"}

    MetricEval -- Yes --> FixLimit["Pod is hitting CPU Limits ceiling.<br/>Increase resources.limits.cpu
    MetricEval -- No --> AppProfile["Application thread contention or external bottleneck"]
```

Diagnostic Explanation:
CPU throttling occurs when a container attempts to burst beyond its CFS quota. Removing or raising CPU limits allows workloads to utilize idle CPU cycles without degrading latency.

58. DiskPressure & Node Eviction: Ephemeral Storage Cleanup

```
flowchart TD
    DiskPress["Node Condition: DiskPressure = True<br/>Pods Evicted with The node was low on resource: ep
    SSH --> CheckDisk["Check Disk Usage: df -h /"]

    CheckDisk --> Step1["1. Prune unused images and exited containers:<br/>crictl rmi --prune"]
    Step1 --> Step2["2. Clean up large container logs:<br/>truncate -s 0 /var/log/pods/*/*/*.log"]
    Step2 --> Step3["3. Inspect /var/lib/kubelet for orphaned pod directories"]

    Step3 --> Recover["Disk utilization drops below threshold (85%) -> Node DiskPressure clears to False"
```

Diagnostic Explanation:
When node disk utilization exceeds 85%, kubelet initiates hard eviction of Pods consuming excessive ephemeral storage. Clearing old container images with `crictl rmi --prune` immediately restores disk headroom.

59. PID Pressure & Kernel Process Exhaustion

```
flowchart TD
    PIDPress["Node Condition: PIDPressure = True"] --> CheckPIDs["SSH to Node & Check Process Count:<br/>

    CheckPIDs --> FindCulprit["Identify container leaking threads/processes:<br/>ps aux --sort -nlwp | he
    FindCulprit --> KillProcess["Drain node & restart runaway container"]

    KillProcess --> EnforceLimit["Add Pod PID Limits in kubelet configuration:<br/>podPidsLimit: 4096 in
```

Diagnostic Explanation:
A fork-bomb or process leak inside a container can exhaust the Linux kernel PID table, causing the entire host to freeze. Modern kubelets enforce `podPidsLimit` to isolate process leaks to the offending container.

60. StatefulSet Out-of-Order Pod Startup & PVC Orphan Cleanup

```
flowchart TD
    StatefulFail["StatefulSet pod db-1 will not start"] --> Step1["Check previous ordinal db-0:<br/>kubec
    Step1 --> IsReady{"Is db-0 in Running & Ready state?"}

    IsReady -- No --> FixDB0["StatefulSets require sequential startup.<br/>Fix db-0 before db-1 can be sc

    IsReady -- Yes --> CheckPVC["Check PVC: kubectl get pvc data-db-1"]
    CheckPVC --> PVCMissing{"Does PVC exist?"}
    PVCMissing -- No --> RecreatePVC["Ensure volumeClaimTemplates is correctly defined in StatefulSet spe
```

Diagnostic Explanation:
By default (`podManagementPolicy: OrderedReady`), StatefulSets require pod $N-1$ to pass all readiness checks before pod N is created. If pod 0 is crashing, pod 1 will never start.

61. Helm Release Stuck in Pending-Install / Pending-Upgrade

```
flowchart TD
    HelmStuck["Helm command fails: release X has another operation in progress"] --> ListReleases["Run: h
    ListReleases --> CheckStatus{"Status: pending-install or pending-upgrade?"}

    CheckStatus -- Yes --> Step1["Identify Helm Secret storing release state:<br/>kubectl get secret -l o
    Step1 --> Step2["Delete the pending secret version:<br/>kubectl delete secret sh.helm.release.v1.<nam
    Step2 --> Step3["Re-run helm rollback or helm upgrade"]
```

Diagnostic Explanation:
Helm 3 stores release state in Kubernetes Secrets within the release namespace. When a deployment is interrupted, the release state gets stuck in `pending-upgrade`. Deleting the hung secret releases the state lock.

62. Custom Resource Definition (CRD) Validation & Operator Errors

```
flowchart TD
    CRDFail["Error creating Custom Resource: validation failure"] --> Step1["Inspect CRD Schema: kubectl
    Step1 --> CheckVersion["Verify apiVersion and kind match CRD spec.group and spec.names"]

    CheckVersion --> Step2["Inspect Operator Logs:<br/>kubectl logs -n <operator-ns> -l app=<operator-nam
    Step2 --> OperatorCrash{"Is operator reconciler throwing panic / errors?"}
    OperatorCrash -- Yes --> RBACCheck["Verify Operator ServiceAccount has RBAC rights to manage the Cust
```

Diagnostic Explanation:
Custom Resources require valid OpenAPI v3 schemas in the CRD definition. If a valid CR is created but never reconciled, inspect the operator pod logs for RBAC authorization failures.

---

63. DaemonSet Pod Missing on Specific Worker Node

```
flowchart TD
    DSMissing["DaemonSet pod not running on worker-02"] --> DescribeNode["Inspect Node: kubectl describe
    DescribeNode --> CheckTaints["Check Taints on worker-02"]

    CheckTaints --> HasCustomTaint{"Does node have custom taints?"}
    HasCustomTaint -- Yes --> AddToleration["Add matching toleration to DaemonSet spec.template.spec.tole

    HasCustomTaint -- No --> CheckSelector["Check nodeSelector & nodeAffinity in DaemonSet spec"]
    CheckSelector --> LabelMatch{"Does worker-02 have required labels?"}
    LabelMatch -- No --> LabelNode["Label node: kubectl label node worker-02 <key>=<value>"]
    LabelMatch -- Yes --> CheckResources["Check if worker-02 is out of CPU or memory"]
```

Diagnostic Explanation:
DaemonSet pods automatically tolerate standard node conditions, but custom operational taints (e.g., `gpu=true:NoSchedule`) or specific `nodeSelectors` will prevent DaemonSet scheduling unless declared.

---

64. Master Exam Day Time-Management & Emergency Task Rescue

```
flowchart TD
    ExamTask["New CKA Exam Task Presented"] --> Step1["1. IMMEDIATELY SWITCH CONTEXT:<br/>kubectl config
    Step1 --> Step2["2. Set Namespace Variable:<br/>NS=<task-namespace> (or append -n $NS to all commands

    Step2 --> AssessTime{"Can this task be completed in under 6 minutes?"}
    AssessTime -- No / Complex & Confusing --> FlagTask["FLAG TASK FOR REVIEW & MOVE TO NEXT TASK IMMEDIA

    AssessTime -- Yes --> ImperativeFirst["Use Imperative Generators:<br/>kubectl create / run ... --dry-
    ImperativeFirst --> ApplyTask["vim task.yaml && kubectl apply -f task.yaml"]
    ApplyTask --> FastVerify["Run Quick Verification:<br/>kubectl get pod/svc -n $NS && curl test"]
    FastVerify --> PassNext["Task Complete -> Proceed to Next Task"]
```

Diagnostic Explanation:
The golden rule of the CKA exam: **Never spend more than 8 minutes on a single task on the first pass**. If a task is unclear, flag it, skip it, and capture all easy points across the remaining tasks before returning.